DEVELOPMENT AND TESTING PLAN

Activity Discovery Platform

A practical implementation roadmap for a mobile-first platform where people discover activities, create plans, request to join, receive host approval, and coordinate in activity-specific groups.

Document version	1.0
Document status	Development baseline
Client	Flutter / Dart mobile application
Backend and data	HTTPS API, PostgreSQL, real-time messaging, push notifications
Related artifacts	SRS, architecture/database diagrams, UML/workflow diagrams, UI/UX design

Core delivery principle: build one complete, testable end-to-end user journey first. Do not build every screen independently before connecting the workflow.

Prepared as an implementation guide. Technology choices can be refined during technical spikes, but business rules and security constraints in this plan should remain enforced.

1. Development objectives and scope

The first release should prove that a person can sign up, select interests, discover an activity, request to join, receive a host decision, and coordinate with approved members. Supporting features are delivered after this core flow works reliably.

MVP scope

- **Identity:** registration, sign-in, sign-out, password reset, session/token handling.
- **Onboarding:** profile basics and selected interests/categories.
- **Activities:** create, edit, publish, browse, search, filter, view details, cancel and complete.
- **Membership:** request to join, withdraw request, host approve/reject, leave activity, capacity enforcement.
- **Coordination:** activity group chat available only to the host and approved members.
- **Notifications:** join request, decision, activity update/cancellation, reminder.
- **Safety:** block/report entry points, privacy-aware location display, basic moderation queue.

Not required for the first MVP unless explicitly added to the SRS

- Payments, subscriptions, public social feed, live video, advanced recommendation ML, complex event ticketing, or full identity verification.
- Do not collect more personal data than necessary to deliver discovery and coordination.

Definition of MVP success

Measure	Release target
Core journey	Registration → interests → discovery → request → host decision → group chat works end-to-end.
Capacity correctness	No activity can exceed its configured confirmed-member capacity under concurrent approvals.
Authorization	Users cannot edit another host’s activity or read chats they are not authorized to access.
Reliability	Critical journey tests pass in CI and on a real Android device before release.
Safety	Block/report flows are reachable and produce auditable server-side records.

2. Recommended technical baseline

Layer	Recommended baseline	Responsibilities
Mobile app	Flutter + Dart	Cross-platform UI, navigation, form validation, app state, push handling.
Backend API	Modular HTTPS REST API	Authentication, authorization, business rules, validation, rate limits.
Database	PostgreSQL	Users, interests, activities, memberships, messages metadata, notifications, reports, blocks.
Real-time chat	WebSocket or managed real-time service	Deliver messages and membership changes to authorized clients.
Push notifications	FCM for Android; add APNs when shipping iOS	Notify users of important state changes; push content should avoid private details.
Media storage	Private object storage if activity/profile images are enabled	Upload validation, file size/type limits, access-controlled URLs.
Operations	CI pipeline, crash reporting, logs, backups	Build validation, diagnosis, deployment, restore readiness.

Backend language/framework decision

Choose one backend framework before implementation and document the decision. The critical requirement is not the language; it is consistent server-side validation, database transactions, authorization, observability, and testability. Keep the API contract independent of Flutter widgets.

Environment separation

- **Local:** developer machine and disposable/test database.
- **Staging:** production-like configuration, synthetic accounts/data, end-to-end testing.
- **Production:** restricted secrets, backups, monitoring, and controlled database migrations.

Architecture rule

Flutter must never connect directly to PostgreSQL or contain database credentials. All trusted business decisions are made by the backend. The client may improve usability with early validation, but server validation is authoritative.

3. Implementation modules and dependencies

Order	Module	Deliverable	Depends on
1	Project foundation	Repository, environments, linting, app shell, routing, API client, error model	None
2	Identity	Register/sign in/out, session refresh, protected routes	Foundation
3	Profile & interests	Onboarding, edit interests, preference storage	Identity
4	Activity lifecycle	Create/edit/publish, details, search/filter, host ownership	Identity, profile
5	Membership workflow	Join request, withdraw, approve/reject, leave, capacity	Activity lifecycle
6	Notifications	In-app notification list and push event integration	Membership, activities
7	Activity chat	Authorized room access, send/read messages, pagination	Membership
8	Safety & moderation	Block/report, moderation records, admin actions	Identity, activities
9	Release hardening	Accessibility, performance, crash monitoring, security review	All MVP modules

Build vertical slices

A vertical slice includes the UI, API endpoint, database operation, authorization checks, loading/error states, and tests. For example, finish ‘request to join’ end-to-end before adding advanced activity recommendations.

Suggested repository structure

```
activity_discovery/
■■■ mobile/ # Flutter app
■ ■■■ lib/core/ # API client, routing, theme, errors
■ ■■■ lib/features/auth/
■ ■■■ lib/features/profile/
■ ■■■ lib/features/activities/
■ ■■■ lib/features/memberships/
■ ■■■ lib/features/chat/
■ ■■■ lib/features/notifications/
■ ■■■ test/ # unit, widget, integration tests
■■■ backend/ # API service
■ ■■■ modules/auth, users, activities, memberships
■ ■■■ modules/chat, notifications, safety
■ ■■■ migrations/
■ ■■■ tests/unit, integration, api
■■■ docs/ # SRS, diagrams, API contract, runbooks
■■■ .github/workflows/ # CI pipelines
```

4. Milestone-based delivery plan

Use milestone exits rather than calendar promises. Each phase is complete only when its deliverables and acceptance checks pass.

Milestone	Work	Exit criteria
M0 — Foundation	Repo, Flutter app shell, backend skeleton, database migration framework, CI	Clean build, lint/static analysis, health endpoint, documented local setup.
M1 — Identity & onboarding	Auth, profile, interests, session handling	New user can register, sign in, select/edit interests, and sign out.
M2 — Activity discovery	Create/publish activity, list/detail, search and filters	Host can publish; other users can discover and view details.
M3 — Join workflow	Request, withdraw, approve/reject, leave, capacity transactions	Every state transition is authorized; concurrent approvals cannot overfill.
M4 — Chat & notifications	Activity chat access control, in-app notifications, push integration	Only approved members/host can access chat; key state changes notify correctly.
M5 — Safety & polish	Report/block, accessibility, empty/error states, privacy review	Safety actions persist; blocked-user rules apply server-side.
M6 — Release candidate	Regression, security, performance, device tests, backups, release notes	No open critical/high defects; rollback and support procedures tested.

Recommended working rhythm

- Plan one small feature slice; define acceptance criteria before coding.
- Create a branch and small reviewable commits; avoid mixing unrelated features.
- Run formatter, static analysis, unit tests, and relevant integration tests before merging.
- Demonstrate the working feature on a device or staging environment at each milestone.
- Keep a bug log with severity, reproduction steps, owner, fix version, and retest result.

5. Backend rules that must be implemented early

These are correctness and security rules, not optional UI behavior.

Rule	Required implementation
Activity ownership	Only the authenticated host (or authorized moderator) may edit, publish, or cancel the activity.
Membership uniqueness	Add a unique database constraint for (activity_id, user_id) so duplicate requests/memberships cannot be created.
Atomic capacity	Approval must recheck confirmed count and update membership in a transaction/locking strategy. Two approvals racing for the last spot must not both succeed.
Membership states	Model requested, approved, rejected, withdrawn/left, and removed explicitly. Define which transitions are allowed.
Chat authorization	Authorize every room join, history fetch, and send operation on the server; never trust client-supplied activity IDs alone.
Blocking	Apply block rules to discovery, joining, messaging, and future contact where applicable; check on the server.
Location privacy	Show approximate location before approval when appropriate; avoid exposing home/private locations or precise coordinates unnecessarily.
Rate limiting	Limit sign-in attempts, join requests, message sending, report submission, and other abuse-prone endpoints.
Audit trail	Record important moderation and membership decisions with actor, timestamp, target, and outcome.
Notifications	Treat notification delivery as retryable; do not let push failure roll back a successful membership transaction.

Example API surface

Area	Example endpoints (illustrative; finalize in API specification)
Auth	POST /auth/register, POST /auth/login, POST /auth/logout, POST /auth/refresh
Profile	GET/PATCH /me, GET/PUT /me/interests
Activities	POST /activities, GET /activities, GET /activities/{id}, PATCH /activities/{id}, POST /activities/{id}/cancel
Memberships	POST /activities/{id}/join, GET /activities/{id}/requests, POST /memberships/{id}/approve, POST /memberships/{id}/reject, POST /activities/{id}/leave
Chat	GET /activities/{id}/messages, POST /activities/{id}/messages; real-time channel auth
Safety	POST /reports, POST /blocks, DELETE /blocks/{userId}

6. Testing strategy

Testing starts with each module and continues through the whole user journey. Automated tests should be repeatable and use isolated test data.

Test level	What to test	When / where
Static checks	Formatter, linter, type analysis, dependency checks	Every pull request
Unit tests	State transitions, validators, filters, capacity policy, mapping and error handling	Every module; fast CI suite
Widget/UI tests	Form validation, buttons, loading/empty/error states, navigation and accessibility labels	Flutter test suite
API integration tests	Authentication, permissions, status codes, validation, persistence, pagination	Backend CI with test database
Database/transaction tests	Unique constraints, rollback, race conditions, migrations	Backend CI; repeat concurrency tests
End-to-end tests	Register → interests → discover → request → approve → chat	Staging and release candidate
Security tests	IDOR/authorization, token handling, injection, rate limits, data leakage, upload abuse	Before beta and each major release
Performance tests	Activity listing, search/filter, chat fan-out, API latency under realistic load	Before release; after architecture changes
Usability/accessibility	Task completion, clear statuses, screen readers, text scaling, contrast, tap target size	Prototype and beta testing
Device/regression tests	Supported Android versions/screen sizes, network loss/retry, app resume, push behavior	Every release candidate

Risk-based priority

- **P0 / critical:** unauthorized access, over-capacity membership, data loss, cross-user chat leakage.
- **P1 / high:** login blocked, host cannot approve, activity cannot be created, key notifications absent.
- **P2 / medium:** filters wrong, layout defects, non-critical notification delay.
- **P3 / low:** minor visual inconsistency or copy issue with a safe workaround.

7. Core test cases and acceptance checks

ID	Scenario	Expected result
TC-01	Register with valid data	Account is created; duplicate email/identity is handled safely; no plaintext password is stored.
TC-02	Submit invalid sign-in details repeatedly	Generic error response; rate limit/lockout policy applied without leaking account existence unnecessarily.
TC-03	Create activity with end time before start time	Validation blocks submission and identifies the field to correct.
TC-04	Unauthenticated user attempts to create activity	Backend returns unauthorized; no activity record is created.
TC-05	Non-host edits or cancels another user's activity	Backend denies the request; original activity remains unchanged.
TC-06	User requests to join a published activity with capacity	One pending request is created; duplicate taps do not create duplicates.
TC-07	Host approves request when a spot remains	Membership becomes approved; count updates; notification is queued; chat access is granted.
TC-08	Two host approvals race for the final spot	At most one approval succeeds; the other receives a clear full/capacity response.
TC-09	User tries to read chat without membership	Backend denies access to history and real-time room; no messages leak.
TC-10	Approved user leaves activity	Membership changes to left; chat access is removed according to policy; capacity is recalculated.
TC-11	Host cancels an activity	Status changes; participants are notified; joining and chat posting follow cancellation policy.
TC-12	A user blocks another user	Block is persisted and enforced by server-side discovery/join/chat rules as defined.
TC-13	Push provider is temporarily unavailable	Core action still succeeds; notification retries or remains in-app; no duplicate membership change.
TC-14	Network drops during form submission	UI shows retryable state; user input is retained; repeated submission is idempotent where applicable.
TC-15	User reports a person/activity	Report is stored with status and audit data; user receives confirmation without revealing internal moderation data.

Release gate

All P0/P1 tests pass; no open critical/high security defect; migrations work on a clean database; restore/rollback instructions are available; and the core journey passes on at least one physical Android device plus staging.

8. CI/CD, environments, and release management

Suggested pull-request pipeline

Stage	Checks
1. Quality	Format, lint/static analysis, unit tests, dependency vulnerability scan.
2. Backend	Unit/API integration tests against an isolated database; migration checks.
3. Mobile	Flutter tests, Android debug build, widget tests.
4. Contract	Validate API schema/examples and error response consistency.
5. Merge	Require green checks and review; protect main branch.
6. Staging	Deploy backend, apply reviewed migrations, install build, run smoke/E2E tests.
7. Release	Tag version, generate signed Android artifact, publish release notes, monitor rollout.

Release checklist

- Use separate production secrets; never commit API keys, signing keys, database passwords, or private service credentials.
- Confirm Android app signing and version/build numbers; verify permissions and privacy disclosures.
- Back up the production database before risky migrations and test restore procedures in advance.
- Use backward-compatible migrations when possible; have a rollback/forward-fix plan.
- Monitor crashes, API error rates, login failures, join/approval failures, notification delivery, and unusual abuse reports.
- Release gradually if the distribution channel supports it; pause rollout if critical metrics regress.

Operational monitoring

Logs should contain correlation/request IDs and useful event context, but avoid passwords, authentication tokens, private messages, precise location, or unnecessary personal information. Define retention and access permissions for logs and reports.

9. First implementation sprint checklist

Use this as the initial working checklist. Complete the items in order and demonstrate each result before moving on.

■ Create the repository and document how to run mobile and backend locally.

■ Set up Flutter formatting, static analysis, test folder, routing, and shared theme.

■ Choose the backend framework; add health endpoint, configuration validation, and API error format.

■ Create PostgreSQL migrations for users, interests, user_interests, activities, memberships, and blocks.

■ Implement authentication and a protected /me endpoint.

■ Build onboarding to save interests and profile basics.

■ Implement activity create/list/detail endpoints and Flutter screens.

■ Implement join request and host decision with transactional capacity enforcement.

■ Add tests for authorization, duplicate requests, and concurrent approvals before adding chat.

■ Implement chat room authorization and message persistence.

■ Add in-app notifications, then push notifications.

■ Add report/block, test privacy rules, and prepare staging smoke tests.

Go / no-go decision

Go: critical user journeys work, P0/P1 tests pass, capacity and authorization tests pass, no critical/high defect is open, monitoring is active, and rollback/restore steps are known.

No-go: chat or activity data can leak across users, capacity can be exceeded, host-only actions lack server checks, production secrets are exposed, or core flows are unreliable.

Document end — keep this plan synchronized with the SRS, API specification, architecture, database schema, and UI/UX designs whenever requirements change.